

VSCode and Conda Environments

VSCode is quite a powerful tool. One of the arguments in programming faster is to minimize use of a mouse and increase use of the keyboard. You will find that VS Code provides an exhaustive list of commands in the Command Palette ( or Ctrl+Shift+P in Windows) so that you can run VS Code without using the mouse. Press  then type a command name (for example 'git') to filter the list of commands.

A few of my colleagues make an active effort to memorize many of the commands VSCode provides; most of these shortcuts with keyboards are slower than using the mouse in the short-term: stick to them and believe that you will get faster in the longer term.



Visual Studio Code

Keyboard shortcuts for Windows

General

Ctrl+Shift+P, F1	Show Command Palette
Ctrl+P	Quick Open, Go to File...
Ctrl+Shift+N	New window/instance
Ctrl+Shift+W	Close window/instance
Ctrl+,	User Settings
Ctrl+K Ctrl+S	Keyboard Shortcuts

Basic editing

Ctrl+X	Cut line (empty selection)
Ctrl+C	Copy line (empty selection)
Alt+ / I	Move line up/down
Shift+Alt+ / I	Copy line up/down
Ctrl+Shift+K	Delete line
Ctrl+Enter	Insert line below
Ctrl+Shift+Enter	Insert line above
Ctrl+Shift+\	Jump to matching bracket
Ctrl+] / I	Indent/outdent line
Home / End	Go to beginning/end of line
Ctrl+Home	Go to beginning of file
Ctrl+End	Go to end of file
Ctrl+ / I	Scroll line up/down
Alt+PgUp / PgDn	Scroll page up/down
Ctrl+Shift+[	Fold (collapse) region
Ctrl+Shift+]	Unfold (uncollapse) region
Ctrl+K Ctrl+[	Fold (collapse) all subregions
Ctrl+K Ctrl+]	Unfold (uncollapse) all subregions
Ctrl+K Ctrl+O	Fold (collapse) all regions
Ctrl+K Ctrl+J	Unfold (uncollapse) all regions
Ctrl+K Ctrl+C	Add line comment
Ctrl+K Ctrl+U	Remove line comment
Ctrl+=	Toggle line comment
Shift+Alt+A	Toggle block comment
Alt+Z	Toggle word wrap

Navigation

Ctrl+T	Show all Symbols
Ctrl+G	Go to Line...
Ctrl+P	Go to File...
Ctrl+Shift+O	Go to Symbol...
Ctrl+Shift+M	Show Problems panel
F8	Go to next error or warning
Shift+F8	Go to previous error or warning
Ctrl+Shift+Tab	Navigate editor group history
Alt+ - / -	Go back / forward

Ctrl+M

Toggle Tab moves focus

Search and replace

Ctrl+F	Find
Ctrl+H	Replace
F3 / Shift+F3	Find next/previous
Alt+Enter	Select all occurrences of Find match
Ctrl+D	Add selection to next Find match
Alt+K Ctrl+D	Move last selection to next Find match
Alt+C / R / W	Toggle case-sensitive / regex / whole word

Multi-cursor and selection

Alt+Click	Insert cursor
Ctrl+Alt+ / I	Insert cursor above / below
Ctrl+U	Undo last cursor operation
Shift+Alt+I	Insert cursor at end of each line selected
Ctrl+L	Select current line
Ctrl+Shift+L	Select all occurrences of current selection
Ctrl+F2	Select all occurrences of current word
Shift+Alt+→	Expand selection
Shift+Alt+←	Shrink selection
Shift+Alt+ +	Column (box) selection
Ctrl+Shift+Alt+ + (drag mouse)	Column (box) selection
Ctrl+Shift+Alt+ + (arrow key)	Column (box) selection
Ctrl+Shift+Alt+ +PgUp/PgDn	Column (box) selection page up/down

Rich languages editing

Ctrl+Space	Trigger suggestion
Ctrl+Shift+Space	Trigger parameter hints
Shift+Alt+F	Format document
Ctrl+K Ctrl+F	Format selection
F12	Go to Definition
Alt+F12	Peek Definition
Ctrl+K F12	Open Definition to the side
Ctrl+,	Quick Fix
Shift+F12	Show References
F2	Rename Symbol
Ctrl+K Ctrl+X	Trim trailing whitespace
Ctrl+K M	Change file language

Editor management

Ctrl+F4, Ctrl+W	Close editor
Ctrl+K F	Close folder
Ctrl+]	Split editor
Ctrl+ / 2 / 3	Focus into 1 st , 2 nd or 3 rd editor group
Ctrl+K Ctrl+ - / -	Focus into previous/next editor group
Ctrl+Shift+PgUp / PgDn	Move editor left/right
Ctrl+K - / -	Move active editor group

File management

Ctrl+N	New File
Ctrl+O	Open File...
Ctrl+S	Save
Ctrl+Shift+S	Save As...
Ctrl+K S	Save All
Ctrl+F4	Close
Ctrl+K Ctrl+W	Close All
Ctrl+Shift+T	Reopen closed editor
Ctrl+K Enter	Keep previous mode editor open
Ctrl+Tab	Open next
Ctrl+Shift+Tab	Open previous
Ctrl+K P	Copy path of active file
Ctrl+K R	Reveal active file in Explorer
Ctrl+K O	Show active file in new window/instance

Display

F11	Toggle full screen
Shift+Alt+O	Toggle editor layout (horizontal/vertical)
Ctrl+= / -	Zoom in/out
Ctrl+B	Toggle Sidebar visibility
Ctrl+Shift+E	Show Explorer / Toggle focus
Ctrl+Shift+F	Show Search
Ctrl+Shift+G	Show Source Control
Ctrl+Shift+D	Show Debug
Ctrl+Shift+X	Show Extensions
Ctrl+Shift+H	Replace in files
Ctrl+Shift+J	Toggle Search details
Ctrl+Shift+U	Show Output panel
Ctrl+Shift+V	Open Markdown preview
Ctrl+K V	Open Markdown preview to the side
Ctrl+K Z	Zen Mode (Esc Esc to exit)

Debug

F9	Toggle breakpoint
F5	Start/Continue
Shift+F5	Stop
F11 / Shift+F11	Step into/out
F10	Step over
Ctrl+K Ctrl+I	Show hover

Integrated terminal

Ctrl+`	Show integrated terminal
Ctrl+Shift+`	Create new terminal
Ctrl+C	Copy selection
Ctrl+V	Paste into active terminal
Ctrl+ / I	Scroll up/down
Shift+PgUp / PgDn	Scroll page up/down
Ctrl+Home / End	Scroll to top/bottom

Other operating systems' keyboard shortcuts and additional unassigned shortcuts available at aka.ms/vscodekeybindings

VSCode has some excellent tutorials. I would recommend going through these first:

1. <https://code.visualstudio.com/docs/getstarted/tips-and-tricks>
2. <https://code.visualstudio.com/docs/editor/codebasics>

It's totally normal to take multiple passes through these tutorials; they are dense, and you are unlikely to use all of the features you encounter regularly. You can return to it as you get comfortable with more and more features to push yourself. If you find yourself repeating a task too many times in a way that feels clunky, you're likely missing a good shortcut for it.

Because we're working in Python, we are going to use the Python extension for VS Code from the Visual Studio Marketplace. The Python extension is named Python and it's published by Microsoft. I would now work through these two tutorials:

1. <https://code.visualstudio.com/docs/python/python-tutorial>
2. <https://code.visualstudio.com/docs/editor/intellisense>

Okay next up, opening a new project on VSCode.

We're going to use the third option, clone git repository. First you will need to go to github, and [fork](#) this repository: <https://github.com/rajpurkar/python-practice>. Then clone your new github repository.

Our colleague needs our help. In *number_of_ways.py*, they have attempted a programming problem unsuccessfully. Now they've asked us to find out what's wrong with the code and help them fix it. Let's go ahead and open *number_of_ways.py*

It's some Python code to solve the leetcode problem:

<https://leetcode.com/problems/number-of-ways-to-reach-a-position-after-exactly-k-steps/>

We can see the programmers note on line 4! Let's see whether we can help our colleague fix this. First things first, we need to get the code to run. And for that, we need conda.

Exercise

Work in pairs to solve the leetcode problem by yourself first. Use VSCode for the editor. You won't need to install any packages.

Conda Environment

We're going to create a conda environment to run the code.

Why an environment? From VSCode's [guide](#):

"By default, any Python interpreter installed runs in its own global environment. They aren't specific to a particular project. For example, if you just run python, python3, or py at a new terminal (depending on how you installed Python), you're running in that interpreter's global environment. Any packages that you install or uninstall affect the global environment and all programs that you run within it.

To prevent such clutter, developers often create a virtual environment for a project. A virtual environment is a folder that contains a copy (or symlink) of a specific interpreter. When you install into a virtual environment, any packages you install are installed only in that subfolder. When you then run a Python program within that environment, you know that it's running against only those specific packages."

What is a conda environment?

"A conda environment is a Python environment that's managed using the conda package manager (see [Getting started with conda \(conda.io\)](#)). Whether to use a conda environment or a virtual one will depend on your packaging needs, what your team has standardized on, etc."

People often get confused about how conda and pip are different from each other. Here's a short summary from the [conda blog](#):

"Conda is a cross platform package and environment manager that installs and manages conda packages from the Anaconda repository as well as from the Anaconda Cloud. Conda packages are binaries. There is never a need to have compilers available to install them. Additionally conda packages are not limited to Python software. They may also contain C or C++ libraries, R packages or any other software.

This highlights a key difference between conda and pip. Pip installs Python packages whereas conda installs packages which may contain software written in any language. For example, before using pip, a Python interpreter must be installed via a system package manager or by downloading and running an installer. Conda on the other hand can install Python packages as well as the Python interpreter directly.

Another key difference between the two tools is that conda has the ability to create isolated environments that can contain different versions of Python and/or the packages installed in them. This can be extremely useful when working with data science tools as different tools may contain conflicting requirements which could prevent them all being installed into a

single environment. Pip has no built in support for environments but rather depends on other tools like virtualenv or venv to create isolated environments. Tools such as pipenv, poetry, and hatch wrap pip and virtualenv to provide a unified method for working with these environments.”

Note that people still use conda and pip together for the following reason:

“A major reason for combining pip with conda is when one or more packages are only available to install via pip. Over 1,500 packages are available in the Anaconda repository, including the most popular data science, machine learning, and AI frameworks. These, along with thousands of additional packages available on Anaconda cloud from channeling including conda-forge and bioconda, can be installed using conda. Despite this large collection of packages, it is still small compared to the over 150,000 packages available on PyPI. Occasionally a package is needed which is not available as a conda package but is available on PyPI and can be installed with pip. In these cases, it makes sense to try to use both conda and pip.”

Go ahead and install Conda if you don't already have it.

<https://docs.conda.io/en/latest/index.html>

You have the option of installing Miniconda or Anaconda:

<https://docs.conda.io/en/latest/miniconda.html>

It doesn't matter which one you use; I use Miniconda. Here's the difference, from [this link](#):

“Miniconda is a free minimal installer for conda. It is a small, bootstrap version of Anaconda that includes only conda, Python, the packages they depend on, and a small number of other useful packages, including pip, zlib and a few others. Use the conda install command to install 720+ additional conda packages from the Anaconda repository.”

Once we have conda, we're going to create a new environment. There are many conda commands; here is the list of the most useful ones (full [pdf](#) link here):

At the least, you should memorize conda activate, conda create, conda env list. My friend creates flashcards to practice memorization of many of these.

It's time to work with conda. We can do this directly from the VSCode terminal – learn how to get started with VSCode terminal by reading the very first section of

<https://code.visualstudio.com/docs/terminal/basics>.

We're first going to create our new environment:

```
conda create -n cs197lec2 python=3.9
```

It's very important that we specify the Python interpreter so that VS Code then shows in the list of available interpreters. Now we can activate the environment from the terminal:

Depending on your settings, we still have to tell VSCode where to find the Python interpreter to use for the program we want to run. We want to use the environment we just created. Use the Command Palette (P or Ctrl+Shift+P on Windows) to select the interpreter. We want the cs197lec2 environment.